RidgeLib

Group and Individual Code Guide

A practical guide to understanding, building, testing, and explaining the C++17 ridge-regression project

Project: RidgeLib (Project 14)

Repository folder: C:/coding c++/Group-7-project-

Guide scope: shared architecture, mathematical conventions, module responsibilities, code walkthroughs, build and tests, and known limitations.

This guide describes the current repository implementation. It does not assert that a particular member authored shared files unless the repository evidence establishes that contribution.

1. What the project does

RidgeLib is a reusable C++17 library for linear and ridge regression. The code is organized as public headers under **include/Group-7-project-ridge-lib**, implementations under **src**, examples under **examples**, and executable tests under **tests**. CMake builds the library target **ridgelib::ridgelib**.

Project flow

```
Numeric CSV -> DataLoader -> explicit missing-feature policy -> split rows
-> fit scaler on training feature(s) -> transform train/validation/test
-> LinearRegression or RidgeRegression / GradientDescent
-> predictions -> RegressionMetrics -> tune lambda on validation data
```

Keep the final test set out of model selection. Fit preprocessing and scaling using training data only, then reuse those fitted parameters for validation and test data. The current split is deterministic and contiguous; it does not shuffle.

Repository map

Path	Purpose
include/Group-7-project-ridge-lib/	Public interfaces and types
src/	C++ implementations compiled into RidgeLib
tests/	CTest programs for modules and workflow
examples/	Runnable demonstrations
reports/	Weekly group and member progress records
output/pdf/	Generated user-facing PDF guide

2. Mathematical conventions

Ridge objective

For n examples and a design matrix X that includes an intercept column, Ridge cost is:

```
J(theta) = [sum_i (X theta - y)_i^2 + lambda * sum_{j=1..p} theta_j^2] / (2n)
```

The first parameter theta[0] is the intercept and is excluded from the penalty. Lambda must be finite and non-negative. RidgeRegression adds the intercept internally; direct ridge_cost and GradientDescent callers must supply a design/parameter convention consistent with their use.

Normal equation and numerical method

```
(X^T X + lambda D) theta = X^T y
D[0,0] = 0; feature diagonal entries = 1
```

The current unregularized normal-equation helper solves X^T X theta = X^T y using Gaussian elimination with partial pivoting. RidgeRegression adds lambda to feature diagonals while leaving the intercept diagonal unchanged. No explicit inverse is computed. Normal equations can still be inaccurate for ill-conditioned designs; feature scaling can help, and QR/SVD would be a stronger future option.

Gradient descent

Batch gradient descent calculates the mean squared-error gradient and adds lambda/n times each non-intercept parameter. The constructor accepts learning rate, max iterations, lambda, and optional absolute cost-change tolerance. Zero tolerance disables early stopping. The result contains parameters, cost history, iteration count, and a convergence flag. A small change in cost is the stopping condition; it does not guarantee a globally well-conditioned optimization problem.

3. Modules and how to read them

Matrix and Vector

Files: matrix.hpp / matrix.cpp
Matrix stores row-major doubles. Public indexing uses operator()(row,col), with bounds checks. It supports transpose, identity, +, -, matrix multiply, scalar multiply, matrix-vector multiply, row/column extraction, and dot. Dimensions are size_t; mismatches throw exceptions.

Dataset loading

Files: data_loader.hpp / data_loader.cpp
Loads numeric CSV. The last CSV column is the target; prior columns are features. Header is optional. Quoted CSV and categorical values are unsupported. Missing features default to reject; optional mean_impute_features computes each column mean from observed values. Missing targets and malformed/non-finite values are rejected.

Preprocessing and scaling

Files: MinMaxScaler.hpp / StandardScaler.hpp
Scalers fit one vector at a time and preserve fitted parameters for transform. MinMax maps constant columns to zero. StandardScaler uses population variance and maps constant columns to zero. Fit only on training data to avoid leakage.

Regression interface and baseline

Files: regressor.hpp / linear_regression.hpp / linear_regression.cpp
Regressor is an abstract runtime-polymorphic interface with virtual fit and predict. LinearRegression implements it and adds an intercept internally. Prediction before fitting and mismatched feature counts throw.

Solver

Files: normal_equation.hpp / normal_equation.cpp
gaussian_solve solves a square system with partial pivoting; fit_normal_equation builds the unregularized normal-equation solution. Singular or non-finite systems are rejected.

Ridge model and cost

Files: ridge_cost.hpp / ridge_regression.hpp / ridge_regression_cost.cpp / ridge_regression.cpp
The model accepts lambda, fits coefficients, adds an intercept internally, and excludes that intercept from regularization. ridge_cost provides the corresponding scalar objective.

Gradient descent

Files: gradient_descent.hpp / gradient_descent.cpp
Fits parameters on a caller-provided design matrix, returning history and convergence metadata. The first design column is assumed to be intercept for penalty exclusion.

Tuning

Files: tuning.hpp / tuning.cpp
GridSearchTuner constructs a model for each candidate, fits on training inputs, scores on validation inputs with R-squared, and returns the best validation model. The test set must not be used as validation.

Metrics

Files: metrics.hpp / metrics.cpp

RegressionMetrics provides MAE, MSE, RMSE, and R-squared. Inputs must be same-size non-empty finite vectors. Constant-target R-squared is defined as 1 for exact predictions and 0 otherwise.

4. Member learning and handoffs

The table identifies responsibility areas from the project merge plan. It is a study and review guide, not proof of individual authorship. Team members should be able to explain the linked header, implementation, related test, a normal case, and an invalid-input case.

Member	Module to explain	Test/example to walk through
Ayesiga Anslem	Matrix/vector; gradient-descent work transferred from Dataset	test_matrix, test_gradient_descent
Wobusobozi Chrispus Owen	CSV loading and split	test_data_loader
Kyelanyi Martin Tom	Min-max and standard scaling; CMake	test_scaler, test_standard_scaler
Kamugisha Elias	Linear-regression baseline	test_linear_regression
Mutyaba Rashid	Regressor interface and prediction contract	test_linear_regression, oop_demo
Khalid Taban Luate	Normal equation / Gaussian solve	test_normal_equation
Mandela Nickson	Ridge cost and model	test_ridge
Ainemugabe Crisbert	Grid-search tuning	test_tuning, oop_demo
Ssemwogerere Othman	Regression metrics	test_metrics
Group responsibility	End-to-end workflow and review	test_workflow, ridge_demo

Suggested module contribution workflow

1. Confirm the public API with dependent owners. 2. Work on a branch named feature/. 3. Add or update the header, implementation, tests, and a small example where useful. 4. Build and run the focused CTest target. 5. Open a pull request with purpose, behavior, tests, and known limitations. 6. A teammate reviews the diff and asks the author to explain the math and edge cases. Follow the group rotation: member 1 reviews 2, continuing cyclically; member 10 reviews member 1.

5. Build, examples, and test commands

Prerequisites: CMake 3.16+ and a C++17 compiler. The examples and tests are enabled by default. Warnings are enabled for the library target.

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --parallel
ctest --test-dir build --output-on-failure
```

Windows with MSYS2 MinGW from PowerShell:

```
cmake -S . -B build-mingw -G "MinGW Makefiles" -DCMAKE_CXX_COMPILER=g++
cmake --build build-mingw --parallel 4
ctest --test-dir build-mingw --output-on-failure
```

Example binaries are ridge_demo and oop_demo. The reusable CMake target is ridgelib::ridgelib. Consumers can use add_subdirectory(path/to/Group-7-project) and target_link_libraries(app PRIVATE ridgelib::ridgelib). Installation/export rules are included in CMake.

Current verification record

The project was configured and built locally with GNU C++ 16.2 using MinGW Makefiles. Both examples ran. Eleven CTest targets passed, including matrix, CSV loading, both scalers, linear regression, normal equation, ridge, gradient descent, tuning, metrics, and an end-to-end scaling/model/metric workflow. Results apply to that local environment and checkout; rerun these commands after changes.

Known limitations

No quoted CSV parser, categorical encoding, randomized split, standard cross-validation, matrix-wise scaler interface, or missing-target imputation. Gradient descent uses cost-delta tolerance, not gradient norm. The solver uses normal equations and may be sensitive to ill-conditioning. Tests are focused and small; fuzzing, sanitizers, large-scale stress tests, and other compiler/platform verification have not been done. The optional external reference comparison was not performed.

A useful workflow example

```
DataLoader loader;
auto data = loader.loadCSV("data.csv", true,
    DataLoader::MissingValuePolicy::mean_impute_features);
auto split = loader.trainTestSplit(data, 0.8);
// Fit scaler(s) on X_train_raw, reuse on validation/test.
RidgeRegression model(0.1);
model.fit(train_matrix, split.y_train);
auto predictions = model.predict(test_matrix);
auto rmse = RegressionMetrics::root_mean_squared_error(
    split.y_test, predictions);
```

The sample shows the interface sequence; it omits conversion helpers between vector-of-rows and Matrix for brevity. Confirm all dimensions at each conversion.

Disclosure: Group members should follow course rules and report assistance honestly. This guide does not claim authorship of members' work.